SERVICENOW SYSTEM ADMINISTRATOR · NAAN MUDHALVAN (SMARTBRIDGE)

Brainstorming & Idea Prioritization

Phase 1 · Brainstorming & Ideation

Deepak P (Team Lead) · Hemanthan B N · Ganesh A K · Irfan Rifayil Z

This document captures the Phase-1 brainstorming session run by Team Nexus for the ServiceNow System Administrator capstone project "Script-Controlled ACL - Restrict Record Access Based on Field Value". The team's goal was to decide how to enforce branch-based, record-level access on the custom u_institution_details table so that a branch user (for example the EEE branch) can only see and manage records whose Branch field matches their branch, on the dev443713 Personal Developer Instance.

How the team brainstormed

- Framed the real problem: out of the box, any user who can reach the table reads and edits every record across all branches (ECE / EEE / CSE), breaking data privacy and record integrity.
- Agreed the success test up front: an EEE branch user must see only EEE records, keep New/Update/Delete on those, and a user with no branch role must be blocked entirely.
- Each member proposed one native-platform mechanism, then the group compared them on how well they enforce the boundary server-side.
- Deliberately excluded any external app, framework, or database - the solution had to be native Now Platform configuration on a free PDI.

Candidate ideas considered

Idea	How it would work	Main limitation
Before/after query Business Rule	A server-side query business rule appends addQuery('u_branch', user's branch) so the list only returns matching rows.	Filters lists but must be re-applied everywhere; a data-restriction concern being solved with scripting rather than the platform's security engine.
Client Script to hide records/fields	A client script hides non-EEE rows or fields in the browser.	Cosmetic only - runs in the browser, is trivially bypassed via URL or API, and provides no real security.
Role-only ACL	A single role on the table grants access by role membership.	All-or-nothing: a role cannot discriminate on the Branch field value, so it cannot restrict to EEE records specifically.
Script-controlled ACL with field condition	Record-level ACLs (one per operation) combine a required role, a data condition (Branch is EEE), and a server-side script (gs.hasRole) evaluated by the ACL engine.	Requires security_admin elevation and careful per-operation setup, but enforces the boundary server-side for UI and API alike.

Idea prioritization

Each idea was scored on Feasibility and Security (1 = poor to 5 = excellent) and on Effort (Low / Medium / High, where lower is better). Security was weighted highest because the project is

fundamentally a data-privacy control.

Idea	Feasibility	Security	Effort	Decision
Query Business Rule	4	3	Medium	Backup only - list-level, not the security engine
Client Script hide	3	1	Low	Rejected - cosmetic, bypassable
Role-only ACL	5	2	Low	Rejected - cannot filter on Branch value
Script-controlled ACL + field condition	5	5	Medium	SELECTED

Selected approach and rationale

- Chosen solution: script-controlled Access Control rules on u_institution_details, one per operation - READ (role bb1, Advanced, condition Branch is EEE, plus a gs.hasRole script), CREATE (bb2), WRITE (bb3), DELETE (bb4).
- It is the only option that is field-value driven: access depends on both the user's role and the record's Branch value, exactly what the problem demands.
- It is enforced by the ACL security engine on the server, so it holds for the list, the form, and direct/API access - not just the visible UI.
- It is native, free, and PDI-friendly: no external components, using sys_security_acl, roles, and server-side GlideScript only.
- Implementation notes captured for the build phase: security_admin elevation is required to edit ACLs, and the default ACLs auto-generated with the u_institution_details_user role are removed so each ACL needs only its bb role.